

Table des Matieres

Diagnostic et Audit Technique	3
1.1 Etat des lieux du depot existant	3
1.2 Matrice des risques techniques	3
1.3 Analyse du chemin critique	4
Decisions d'Architecture	4
2.1 Architecture cible : Modular Monolith	4
2.2 Structure de dossiers cible	5
2.3 Strategie d'etat (4 couches)	5
2.4 Design patterns API	6
2.5 Pyramide de tests	6
2.6 Pipeline CI/CD	6
Roadmap Detaillee	6
3.1 Phase 0 : Fondations et Audit (Semaines 1-2, 10 jours)	6
3.1.1 Epic 1 : Audit et Configuration Projet (5 jours)	7
3.1.2 Epic 2 : Infrastructure Qualite (5 jours)	7
3.2 Phase 1 : Architecture Noyau (Semaines 3-5, 15 jours)	7
3.2.1 Epic 1 : Layout et Navigation (8 jours)	7
3.2.2 Epic 2 : Design System et Composants (7 jours)	8
3.3 Phase 2 : Couche Donnees et API (Semaines 6-7, 10 jours)	8
3.3.1 Epic 1 : Base de donnees (5 jours)	8
3.3.2 Epic 2 : API Routes (5 jours)	8
3.4 Phase 3 : Editeur Central et Workspace (Semaines 8-10, 15 jours)	9
3.4.1 Epic 1 : Editeur Tiptap (8 jours)	9
3.4.2 Epic 2 : Workspace et Chapitres (7 jours)	9
3.5 Phase 4 : Moteur IA et Assistant (Semaines 11-13, 15 jours)	10
3.5.1 Epic 1 : Infrastructure IA (5 jours)	10
3.5.2 Epic 2 : Modes d'ecriture IA (5 jours)	10

3.5.3 Epic 3 : Chat et Notebook (5 jours)	10
3.6 Phase 5 : Modules Academiques (Semaines 14-16, 15 jours)	11
3.6.1 Epic 1 : Methodologie et Articles (8 jours)	11
3.6.2 Epic 2 : Bibliographie et Plan (7 jours)	11
3.7 Phase 6 : Modules Avances (Semaines 17-19, 15 jours)	11
3.7.1 Epic 1 : Outils IA avances (5 jours)	11
3.7.2 Epic 2 : Outils collaboratifs (5 jours)	12
3.7.3 Epic 3 : Integrations et Recherche (5 jours)	12
3.8 Phase 7 : Qualite, Securite et Production (Semaines 20-21, 10 jours)	12
3.8.1 Epic 1 : Qualite et Tests (5 jours)	12
3.8.2 Epic 2 : Securite et Deploiement (5 jours)	13
Specifications Techniques	13
4.1 Design System	13
4.2 Cibles de Performance	14
4.3 Contrats API	14
Gouvernance et Indicateurs	14
5.1 KPIs par phase	14
5.2 Format de Sprint Review	15
5.3 Calendrier synthetique	15
Budget et Ressources	15
6.1 Effort total	16
6.2 Duree et jalons	16

Diagnostic et Audit Technique

Cette section presente l'etat des lieux du depot ThesisFrame existant, identifie les risques techniques majeurs et analyse le chemin critique vers une reconstruction professionnelle. Le depot actuel (v0.4.0) presente une architecture monolithique complexe qui necessite une refonte complete pour atteindre les standards de qualite requis pour un outil doctoral.

1.1 Etat des lieux du depot existant

Le depot GitHub **freemind25/these-frame** dans sa version v0.4.0 revele une architecture typique de prototype rapide qui a atteint ses limites structurelles. L'analyse statique du code met en evidence les problematiques suivantes qui compromettent la maintenabilite et l'evolutivite du projet a moyen terme.

Architecture monolithique critique : Le fichier page.tsx principal contient environ 900 lignes de code avec plus de 30 etats React (useState) integres directement, rendant le composant impossible a tester unitairement et extremement difficile a maintenir. Chaque modification risque des effets de bord non maitrises.

Composants et routes surdimensionnes : 41 composants metier coupls et plus de 40 routes API sans validation consistante forment un reseau de dependances inextricable. L'absence de separation claire entre la logique metier et la couche de presentation rend chaque evolution hasardeuse.

Modeles de donnees complexes : 20+ modeles Prisma definissent une structure de donnees academique riche (theses, chapitres, references, cadrage, notebook, versions). Ce schema constitue un actif precieux pour la reconstruction mais necessite une migration soigneusement planifiee.

Contenu academique substantiel : 26 fichiers de donnees contenant environ 6 240 lignes de contenu academique structure (guides methodologiques, templates de theses, references bibliographiques) representent un capital connaissances qu'il est imperatif de preserver lors de la reconstruction.

Absence totale de tests : Aucun test unitaire, d'integration ou end-to-end n'existe dans le depot. La variable ignoreBuildErrors est activee a true dans la configuration Next.js, masquant les erreurs TypeScript et rendant le type checking ineffectif.

1.2 Matrice des risques techniques

L'analyse des risques identifie huit risques majeurs qui doivent etre adresses de maniere proactive tout au long du projet de reconstruction. Chaque risque est evalue selon sa probabilite d'occurrence et son impact potentiel sur le succes du projet, permettant ainsi de prioriser les actions d'attenuation de maniere coherente.

ID	Risque	Probabilite	Impact	Niveau	Attenuation
R1	Dette technique du monolithe	Elevee	Critique	Majeur	Refonte modulaire P0-P1
R2	Absence de tests	Certaine	Eleve	Critique	Infrastructure Vitest P0

ID	Risque	Probabilite	Impact	Niveau	Attenuation
R3	TypeScript non verifie	Certaine	Moyen	Majeur	ESLint strict P0
R4	Dependances non maitrisees	Moyenne	Eleve	Majeur	Audit + lockfile P0
R5	Performance UI monolithe	Elevee	Moyen	Eleve	Code splitting P3
R6	Securite cles API localStorage	Moyenne	Critique	Majeur	Server-side vault P7
R7	Evolutivite limitee	Certaine	Eleve	Critique	Architecture modulaire P1
R8	Documentation absente	Certaine	Moyen	Majeur	Docs techniques P7

1.3 Analyse du chemin critique

Le chemin critique de la reconstruction passe par quatre jalons incontournables qui conditionnent le succes global du projet. Premierement, la mise en place de l'infrastructure de qualite (lint, tests, CI/CD) constitue le fondement sans lequel aucune evolution ulterieure ne peut etre validee de maniere fiable. Deuxiemement, la definition de l'architecture modulaire et l'implementation du layout principal etablissent le squelette structurel sur lequel tous les modules viendront s'articuler.

Troisiemement, la couche de donnees et les API routes constituent le pont entre le schema Prisma existant et l'interface utilisateur, necessitant une migration soignee des 20+ modeles. Quatriemement, l'editeur central Tiptap avec ses extensions avancees represente le composant le plus complexe du systeme et le veritable coeur fonctionnel de l'application. Le retard sur l'un de ces quatre jalons se repercute directement sur la livraison totale du projet.

L'estimation totale du projet s'eleve a environ 315 jours-homme repartis sur 21 semaines (5 mois), avec une equipe impliquant un lead developpeur a temps plein, un designer UX/UI a mi-temps, un testeur QA a quart temps et un expert IA/NLP a quart temps.

Decisions d'Architecture

Cette section detaille les choix architecturaux fondamentaux qui guideront la reconstruction de ThesisFrame. Chaque decision est argumentee en fonction du contexte specifique du projet (equipe de 1 a 3 developpeurs, application monopage complexe, contraintes de temps) pour eviter toute sur-ingenierie tout en preservant l'evolutivite a long terme.

2.1 Architecture cible : Modular Monolith

L'architecture cible retenue est celle du **Monolithe Modulaire** suivant le paradigme **Feature-Sliced Design**. Ce choix strategique s'appuie sur une analyse comparative rigoureuse des alternatives disponibles et des contraintes specifiques du projet. Contrairement aux micro-frontends qui introduiraient une complexite

operationnelle disproportionnée pour une équipe de 1 à 3 développeurs, le monolithe modulaire offre le meilleur équilibre entre séparation des responsabilités et simplicité de déploiement.

- **Simplicité opérationnelle** : Un seul dépôt, un seul pipeline CI/CD, un seul processus de déploiement. Pas de coordination inter-équipes nécessaire.
- **Partage d'état simplifié** : Les modules communiquent directement via des imports TypeScript, sans API inter-service ni event bus complexe.
- **Refactoring aisé** : Un seul codebase permet les refactoring cross-modules en une seule opération, alors que les micro-frontends figent les frontières.
- **Performance optimale** : Pas de sur-réseau ni de sérialisation/désérialisation entre services. Le bundler optimise globalement.
- **Evolutive future** : Si le besoin se présente, l'extraction d'un module en micro-frontend est facilitée par la séparation claire des features.

2.2 Structure de dossiers cible

La structure de dossiers suit le paradigme Feature-Sliced Design avec une séparation nette entre les modules métier, les composants UI génériques, la logique partagée et les données. Chaque module est autonome et expose uniquement une API publique via un fichier `index.ts` barrel export, garantissant l'encapsulation et la maintenabilité.

```
src/
app/
  layout.tsx / page.tsx (orchestrateur léger)
  api/ (health, thesis, editor, ai, references, methodology)
modules/
  thesis/ components/ hooks/ api/ types/ index.ts
  editor/ ai-writing/ references/ methodology/ notebook/
components/ (ui/ shadcn + layout/)
lib/ (db, ai, utils)
data/ types/
```

2.3 Stratégie d'état (4 couches)

La gestion d'état adopte une approche en quatre couches complémentaires, chacune dédiée à un type de données spécifique pour éviter le mélange des responsabilités et garantir la cohérence de l'application. Cette stratification permet de choisir l'outil le plus adapté à chaque besoin sans compromettre la performance ou la maintenabilité.

Couche	Technologie	Responsabilité	Exemple
Server State	TanStack Query	Données serveur cachées	Liste des thèses, références
Module Store	Zustand	État métier par module	Editeur actif, mode IA courant

Couche	Technologie	Responsabilite	Exemple
UI Local	useState/useReducer	Etat composant local	Modal ouvert, toggle sombre
URL State	searchParams	Etat navigable	Onglet courant, filtre actif

2.4 Design patterns API

Les API routes suivent un pattern RESTful standardise avec validation Zod a l'entree et un format de reponse uniforme. Chaque route implemente un schema de reponse au format { **data**, **error**, **meta** } avec les codes HTTP appropriatifs (200, 201, 400, 401, 404, 500). Le rate limiting protege les endpoints sensibles et un error handler centralise garantit des reponses coherentes en cas d'erreur. La pagination utilise un systeme cursor-based pour optimiser les performances sur les grands jeux de donnees bibliographiques.

2.5 Pyramide de tests

La strategie de tests suit une pyramide classique avec trois niveaux complementaires. Les tests unitaires avec Vitest visent une couverture de 80%, constituant la base de la qualite logicielle. Les tests d'integration utilisant Vitest avec MSW (Mock Service Worker) valident les interactions entre composants et API. Les tests end-to-end avec Playwright couvrent 15 scenarios critiques representatifs des parcours utilisateur principaux.

2.6 Pipeline CI/CD

Le pipeline d'integration et de deploiement continu suit une sequence stricte en six etapes : **Lint (ESLint)** puis **Type Check (tsc --noEmit)** puis **Unit Tests (Vitest)** puis **Build (next build)** puis **E2E Tests (Playwright)** puis **Deploy (Vercel/Docker)**. Chaque etape bloque la suivante en cas d'echec, garantissant que seul du code valide et teste atteint la production.

Roadmap Detaillee

Cette section presente le planning detaille de la reconstruction en huit phases (P0 a P7), couvrant 21 semaines de developpement soit environ 315 jours-homme. Chaque phase est decomposee en epics avec des taches individuelles, des estimations d'effort et des livrables clairement definis. Les criteres d'acceptation de chaque phase garantissent la qualite et la completude avant de passer a la phase suivante.

3.1 Phase 0 : Fondations et Audit (Semaines 1-2, 10 jours)

Objectifs : Auditer le code existant et documenter les risques, mettre en place l'infrastructure de qualite, configurer le pipeline CI/CD, definir le design system.

3.1.1 Epic 1 : Audit et Configuration Projet (5 jours)

#	Tache	Description	Effort	Livable
0.1.1	Cloner et analyser le depot	Analyse statique du code existant	1j	Rapport d'audit
0.1.2	Configurer ESLint strict	Regles TypeScript strict, pas d'any	0.5j	eslint.config.mjs
0.1.3	Activer le type checking	Retirer ignoreBuildErrors	0.5j	next.config.ts propre
0.1.4	Definir conventions de code	Naming, formatting, structure	1j	.editorconfig + guide
0.1.5	Mettre a jour package.json	Nom, version, scripts utiles	0.5j	package.json
0.1.6	Configurer variables env	Schema validation des env vars	1j	.env.example + schema
0.1.7	Documenter l'audit	Synthese des findings	0.5j	AUDIT.md

3.1.2 Epic 2 : Infrastructure Qualite (5 jours)

#	Tache	Description	Effort	Livable
0.2.1	Installer Vitest	Setup avec coverage	1j	vitest.config.ts
0.2.2	Installer Playwright	Setup E2E avec baseURL	1j	playwright.config.ts
0.2.3	Pipeline GitHub Actions	Lint + test + build + E2E	1.5j	.github/workflows/
0.2.4	Configurer Husky + lint-staged	Pre-commit hooks	0.5j	.husky/
0.2.5	Definir le design system	Tokens couleur, typo, espacement	1.5j	tokens.ts + storybook
0.2.6	Creer composants de base	Button, Card, Input, Badge (audit)	1j	Composants verifies

Criteres d'acceptation : Pipeline CI vert, couverture $\geq 30\%$, 0 erreurs TypeScript, design system documente.

3.2 Phase 1 : Architecture Noyau (Semaines 3-5, 15 jours)

Objectifs : Implementer le layout principal responsive, creer le systeme de navigation, developper les composants layout, etablir la structure modulaire.

3.2.1 Epic 1 : Layout et Navigation (8 jours)

#	Tache	Description	Effort	Livable
1.1.1	Layout racine responsive	Header, Sidebar, Main, Footer sticky	2j	RootLayout
1.1.2	Sidebar de navigation	Menu collapsible, breadcrumbs	2j	Sidebar component

#	Tache	Description	Effort	Livrable
1.1.3	Système de routing	Client-side tabs + deep linking	2j	Navigation system
1.1.4	Page d'accueil dashboard	Vue d'ensemble avec stats	2j	Dashboard page

3.2.2 Epic 2 : Design System et Composants (7 jours)

#	Tache	Description	Effort	Livrable
1.2.1	Theme clair/sombre	next-themes + CSS variables	1j	ThemeProvider
1.2.2	Composants métier de base	ChapterCard, StatusBadge, ProgressBar	2j	Base components
1.2.3	Système de modales/dialogs	Composables réutilisables	1.5j	Dialog system
1.2.4	Toasts et notifications	Feedback utilisateur standardisé	0.5j	Toast system
1.2.5	Loading states	Skeletons, spinners, empty states	1j	Loading components
1.2.6	Composables partagés	useDebounce, useLocalStorage, useMediaQuery	1j	Hooks library

***Critères d'acceptation :** Navigation fonctionnelle, responsive mobile/desktop, thème clair/sombre, tous composants testés unitairement.*

3.3 Phase 2 : Couche Données et API (Semaines 6-7, 10 jours)

Objectifs : Implémenter le schéma Prisma complet, développer les API routes CRUD, mettre en place la logique métier serveur avec validation et error handling standardisés.

3.3.1 Epic 1 : Base de données (5 jours)

#	Tache	Description	Effort	Livrable
2.1.1	Schema Prisma Thesis	Modèle Thèse avec chapitres	1j	schema.prisma
2.1.2	Schema Prisma References	CRUD références bibliographiques	1j	schema.prisma
2.1.3	Schema Cadrage et Versions	Modèle cadrage avec versioning	1j	schema.prisma
2.1.4	Schema Notebook et Sources	Notebook de recherche	0.5j	schema.prisma
2.1.5	Seed data	Données initiales de démonstration	1j	seed.ts
2.1.6	Migrations et validation	db:push, vérification intégrité	0.5j	Base opérationnelle

3.3.2 Epic 2 : API Routes (5 jours)

#	Tache	Description	Effort	Livable
2.2.1	API Thesis CRUD	GET/POST/PUT/DELETE theses	1j	/api/thesis/
2.2.2	API Chapters CRUD	Gestion des chapitres	1j	/api/thesis/chapters/
2.2.3	API References	CRUD + recherche + BibTeX export	1.5j	/api/references/
2.2.4	API Cadrage	Sauvegarde et versions	1j	/api/cadrage/
2.2.5	Validation Zod	Schemas de validation pour toutes routes	0.5j	api-schemas.ts

Criteres d'acceptation : Toutes les API testees (integration tests), schema DB stable, CRUD fonctionnel, validation Zod sur toutes les routes.

3.4 Phase 3 : Editeur Central et Workspace (Semaines 8-10, 15 jours)

Objectifs : Integrer l'editeur Tiptap avec extensions avancees, developper le workspace de these, implementer la gestion des chapitres avec drag and drop, auto-save et historique des versions.

3.4.1 Epic 1 : Editeur Tiptap (8 jours)

#	Tache	Description	Effort	Livable
3.1.1	Setup Tiptap core	Installation + extensions de base	1j	TiptapEditor component
3.1.2	Toolbar complete	Gras, italique, titres, listes, liens	2j	EditorToolbar
3.1.3	Extensions avancees	Highlight, typography, character count	1j	Extensions config
3.1.4	Menu IA inline	Menu contextuel sur selection texte	2j	InlineAIMenu
3.1.5	Auto-save	Debounced save + indicateur de statut	1j	useAutoSave hook
3.1.6	Mode fullscreen et focus	Zen mode, split view	1j	Editor modes

3.4.2 Epic 2 : Workspace et Chapitres (7 jours)

#	Tache	Description	Effort	Livable
3.2.1	Onglets de chapitres	Navigation horizontale drag and drop	2j	ChapterTabs
3.2.2	Panneau lateral outils	Tools sidebar avec categories	1.5j	ToolsSidebar
3.2.3	Header de chapitre	Titre editable, statut, mot count	1j	ChapterHeader
3.2.4	Gestionnaire de these	Creer/charger/sauvegarder these	1.5j	ThesisManager
3.2.5	Templates de these	10 templates de structure	1j	TemplateSystem

Criteres d'acceptation : Editeur fonctionnel avec 10+ extensions, auto-save operationnel, drag and drop chapitres, templates applicables et testes.

3.5 Phase 4 : Moteur IA et Assistant (Semaines 11-13, 15 jours)

Objectifs : Implementer le routeur IA multi-fournisseur, developper les 10 modes d'ecriture IA, creer le chat directeur, construire le notebook de recherche avec graphe de connaissances.

3.5.1 Epic 1 : Infrastructure IA (5 jours)

#	Tache	Description	Effort	Livable
4.1.1	Wrapper Z.ai SDK	zai.ts avec retry logic et queue	1j	lib/zai.ts
4.1.2	Routeur multi-provider	9 providers avec fallback	2j	lib/ai-router.ts
4.1.3	Conversation store	Persistence conversations	1j	conversation-store.ts
4.1.4	UI selection provider	Dialog configuration provider	1j	ProviderSettings

3.5.2 Epic 2 : Modes d'ecriture IA (5 jours)

#	Tache	Description	Effort	Livable
4.2.1	Mode Redaction scientifique	System prompt + UI	1j	Mode 1
4.2.2	Mode Revue de litterature	System prompt + UI	1j	Mode 2
4.2.3	Mode Peer review	System prompt + UI	0.5j	Mode 3
4.2.4	Mode Paraphrase	System prompt + UI	0.5j	Mode 4
4.2.5	Modes restants (6)	Abstract, hypothese, methodo, theorie, supervision, soutenance	2j	Modes 5-10

3.5.3 Epic 3 : Chat et Notebook (5 jours)

#	Tache	Description	Effort	Livable
4.3.1	Chat directeur IA	Simulateur de directeur de these	2j	DirecteurChat
4.3.2	Notebook de recherche	Notes, sources, Q&A;	1.5j	ResearchNotebook
4.3.3	Graphe de connaissances	Force-directed SVG	1.5j	KnowledgeGraph

Criteres d'acceptation : 10 modes IA fonctionnels, chat directeur operationnel, notebook avec graphe de connaissances, fallback provider automatique.

3.6 Phase 5 : Modules Academiques (Semaines 14-16, 15 jours)

Objectifs : Developper les guides methodologiques complets, implementer la gestion bibliographique, creer le generateur de plan LaTeX, ajouter les outils academiques et les bases de donnees de references.

3.6.1 Epic 1 : Methodologie et Articles (8 jours)

#	Tache	Description	Effort	Livable
5.1.1	Guide methodologique (7 onglets)	Approche, problematique, variables, collecte, docs, concepts, test	3j	MethodologyTab
5.1.2	Guide articles scientifiques (7 onglets)	Etapes, IMRaD, erreurs, checklist, toolbox, revue syst., soutenance	3j	ArticlesTab
5.1.3	Contenu academique	Migration des 26 fichiers de donnees	2j	data/ complet

3.6.2 Epic 2 : Bibliographie et Plan (7 jours)

#	Tache	Description	Effort	Livable
5.2.1	References bibliographiques	CRUD, tags, recherche, filtres	2j	ReferencesTab
5.2.2	Export BibTeX	Generation et telechargement	1j	BibTeX export
5.2.3	Generateur plan LaTeX	Template personnalisable + download .tex	2j	ThesisPlanTab
5.2.4	Bases de donnees academiques	7 ressources cataloguees	1j	AcademyDB
5.2.5	SLR Protocol	Protocole de revue systematique	1j	SLRProtocolPanel

Criteres d'acceptation : 14 onglets academiques fonctionnels, CRUD references operationnel, export BibTeX fonctionnel, generation LaTeX valide.

3.7 Phase 6 : Modules Avances (Semaines 17-19, 15 jours)

Objectifs : Implementer les outils IA avances, ajouter Excalidraw et la visualisation de donnees, developper les integrations cloud et la reconnaissance vocale, creer la roadmap agile interactive.

3.7.1 Epic 1 : Outils IA avances (5 jours)

#	Tache	Description	Effort	Livable
6.1.1	Humanizer	Humanisation de texte IA	1j	Humanizer

#	Tache	Description	Effort	Livable
6.1.2	Consensus	Analyse multi-sources	2j	Consensus
6.1.3	Visualisation de donnees	6 types de graphiques SVG	1.5j	DataViz
6.1.4	APA Results Composer	Redaction resultats APA	1.5j	APAComposer

3.7.2 Epic 2 : Outils collaboratifs (5 jours)

#	Tache	Description	Effort	Livable
6.2.1	Excalidraw	Tableau blanc dessin	2j	ExcalidrawTab
6.2.2	Roadmap agile	Sprints, stories, phases 0-4	1.5j	AgileRoadmap
6.2.3	ASR dictée vocale	Speech-to-text	1.5j	DictationButton

3.7.3 Epic 3 : Integrations et Recherche (5 jours)

#	Tache	Description	Effort	Livable
6.3.1	Recherche bibliographique	Recherche web integree	1.5j	LiteratureSearch
6.3.2	Journal Finder	Trouver des journaux adaptes	1.5j	JournalFinder
6.3.3	Cloud drive backup	Google Drive, OneDrive	1.5j	CloudDriveBackup
6.3.4	Fiches de lecture	Problematiche, methodes, resultats	1.5j	RecherchePanel

***Criteres d'acceptation :** Tous les modules avances fonctionnels, Excalidraw integre, ASR operationnel, tools IA avec fallback.*

3.8 Phase 7 : Qualite, Securite et Production (Semaines 20-21, 10 jours)

Objectifs : Atteindre 80% de couverture de tests, securiser l'application, preparer le deploiement production, documenter le projet de maniere exhaustive.

3.8.1 Epic 1 : Qualite et Tests (5 jours)

#	Tache	Description	Effort	Livable
7.1.1	Tests unitaires manquants	Atteindre 80% couverture	2j	Tests unitaires
7.1.2	Tests E2E Playwright	15 scenarios critiques	1.5j	Tests E2E
7.1.3	Tests d'integration API	Toutes les routes API	1j	Tests integration

#	Tache	Description	Effort	Livrable
7.1.4	Performance audit	Lighthouse, bundle analysis	0.5j	Rapport perf

3.8.2 Epic 2 : Securite et Deploiement (5 jours)

#	Tache	Description	Effort	Livrable
7.2.1	Securiser les API	Rate limiting, input sanitization	1j	API securisee
7.2.2	Gestion cles API	Server-side key vault	1j	Key management
7.2.3	Configuration deploiement	Docker + Vercel config	1.5j	Deploy config
7.2.4	Documentation technique	README, CONTRIBUTING, API docs	1j	Documentation
7.2.5	Checklist de release	Verification finale	0.5j	Release ready

Criteres d'acceptation : Couverture >= 80%, 15 scenarios E2E passants, zero vulnerabilite critique, documentation complete.

Specifications Techniques

Cette section definit les standards techniques que l'application ThesisFrame doit respecter en termes de design system, performance, contrats API et conventions de developpement. Ces specifications constituent le cadre de reference pour l'ensemble des phases de developpement.

4.1 Design System

Le design system repose sur trois piliers fondamentaux : la palette de couleurs avec variables CSS proposant 10 nuances derivees d'une teinte principale, la typographie Geist (sans-serif) avec 6 niveaux de titrage (H1 a H6) et un systeme d'espacement base sur une grille de 4px (4, 8, 12, 16, 24, 32, 48 pixels). L'ecosysteme de composants comprend plus de 60 composants dont 50 issus de shadcn/ui et 10 composants custom developpes specifiquement pour les besoins academiques de ThesisFrame.

Composant	Standard	Details
Palette couleurs	10 nuances CSS variables	Derivees d'une teinte principale
Typographie	Geist Sans (6 niveaux)	H1: 36pt, H2: 24pt, H3: 18pt, Body: 14pt
Espacement	Grille 4px	4, 8, 12, 16, 24, 32, 48px
Composants UI	60+ composants	50 shadcn/ui + 10 custom
Icones	Lucide React	Suite de 1000+ icones SVG

Composant	Standard	Details
Animations	Framer Motion	Transitions subtiles, respects prefers-reduced-motion

4.2 Cibles de Performance

Les objectifs de performance suivent les recommandations Core Web Vitals avec des seuils stricts pour garantir une experience utilisateur fluide. Les cibles incluent un Largest Contentful Paint inferieur a 2.5 secondes, un First Input Delay inferieur a 100 millisecondes, et un Cumulative Layout Shift inferieur a 0.1. Le bundle JavaScript initial doit rester sous 250 Ko avec des chunks de route inferieurs a 500 Ko. Le temps de reponse API (p95) ne doit pas depasser 500 millisecondes.

Metrique	Cible	Mesure
LCP (Largest Contentful Paint)	< 2.5s	Core Web Vitals
FID (First Input Delay)	< 100ms	Core Web Vitals
CLS (Cumulative Layout Shift)	< 0.1	Core Web Vitals
Bundle initial	< 250KB	Code splitting + tree shaking
Chunk par route	< 500KB	Lazy loading dynamique
API response (p95)	< 500ms	Monitoring edge/server

4.3 Contrats API

Toutes les API routes adherent a un format de reponse uniforme structure en trois champs : **data** (la donnee retournee), **error** (les informations d'erreur si applicable) et **meta** (les metadonnees de pagination et de requete). Les codes HTTP utilises sont strictement 200 (succes), 201 (cree), 400 (erreur client), 401 (non autorise), 404 (non trouve) et 500 (erreur serveur). La validation utilise des schemas Zod definis pour chaque route, et la pagination suit un modele cursor-based pour optimiser les performances sur les grands jeux de donnees bibliographiques.

Gouvernance et Indicateurs

Cette section etablit le cadre de gouvernance du projet avec les KPIs de suivi par phase, le format des revues de sprint et le calendrier synthetique. Ces indicateurs permettent de mesurer objectivement la progression et la qualite du projet tout au long des 21 semaines de developpement.

5.1 KPIs par phase

Chaque phase dispose d'indicateurs clés de performance mesurables qui permettent de valider l'atteinte des objectifs qualitatifs et quantitatifs. La couverture de tests, le nombre de bugs critiques, le score de performance Lighthouse et le débit de développement en points par story constituent les quatre axes de mesure principaux.

KPI	P0 Cible	P2 Cible	P4 Cible	P7 Cible
Couverture de tests	>= 30%	>= 50%	>= 65%	>= 80%
Bugs critiques	0	0	<= 2	0
Score Lighthouse	>= 85	>= 90	>= 90	>= 95
Débit (points/story)	8	10	12	10

5.2 Format de Sprint Review

Les revues de sprint suivent un format structuré en quatre étapes. La **démonstration fonctionnelle** présente les fonctionnalités terminées en conditions réelles. La **revue de code** examine la qualité du code produit et le respect des conventions. Les **métriques qualité** sont présentées et comparées aux cibles définies. L'**ajustement de la roadmap** permet de réorienter les priorités si nécessaire en fonction des résultats observés.

5.3 Calendrier synthétique

Phase	Période	Durée	Effort (j/h)	Focus Principal
P0	S1-S2	10 jours	10	Fondations, CI/CD, design system
P1	S3-S5	15 jours	15	Layout, navigation, composants
P2	S6-S7	10 jours	10	Schema Prisma, API routes CRUD
P3	S8-S10	15 jours	15	Editeur Tiptap, workspace
P4	S11-S13	15 jours	15	Moteur IA, 10 modes, chat
P5	S14-S16	15 jours	15	Methodologie, bibliographie, LaTeX
P6	S17-S19	15 jours	15	Outils IA, Excalidraw, integrations
P7	S20-S21	10 jours	10	Qualité, sécurité, production

Budget et Ressources

Cette section présente le budget global du projet en termes d'effort, de durée et de profils requis. L'estimation prend en compte la complexité de la reconstruction modulaire, la richesse fonctionnelle de l'application cible et les contraintes de qualité inhérentes à un projet de cette envergure.

6.1 Effort total

L'effort total estime pour la reconstruction complete de ThesisFrame s'eleve a **environ 315 jours-homme**. Cette estimation repartit sur **21 semaines** (soit 5 mois) de travail effectif, en tenant compte d'un rythme soutenu mais realiste pour une equipe de developpement agile. La marge d'incertitude est estimee a +/- 15%, ce qui represente un ecart de 47 jours-homme, pris en compte dans le planning par des buffers integres a chaque phase.

Ressource	Implication	Role Principal	Phases Actives
Lead Developer	100%	Architecture, dev frontend/backend, revue de code	P0 a P7
Designer UX/UI	50%	Design system, mockups, tests utilisateurs	P0, P1, P3, P5
Testeur QA	25%	Tests E2E, tests d'integration, reporting	P0, P7
Expert IA/NLP	25%	System prompts, evaluation modes IA	P4, P5

6.2 Duree et jalons

La duree totale du projet est de 21 semaines (5 mois) avec 8 jalons de livraison correspondant aux phases P0 a P7. Chaque jalon est associe a des criteres d'acceptation stricts qui doivent etre validates avant de demarrer la phase suivante. Les trois premiers jalons (P0-P2) constituent le socle technique sur lequel repose l'ensemble du projet. Les phases P3 et P4 representent le coeur fonctionnel avec l'editeur et le moteur IA. Les phases P5 a P7 completent l'offre fonctionnelle et assurent la qualite production.